arXiv:2606.10832v2 [cs.RO] 24 Sep 2026

GUIDE: Goal-Initialized Directional Understanding for End-to-End Legged Navigation

Liang Wang[1,2,*], Jin Jin[3,*], Kanzhong Yao[2], Yibin Wu[3,4], Fangqiang Ding[5], Jin Wang[3], Jun Wu[1], Zhe Sun[2,†], Qiuguo Zhu[1,†]

***Abstract*— End-to-end reinforcement learning (RL) has shown strong potential for legged robot navigation, yet existing approaches commonly rely on continuously updated robot-to-goal states from external state estimation modules, leaving part of the navigation problem outside the learned policy. In this work, we seek to push the limits of end-to-end sim-to-real RL navigation by investigating whether a legged robot can internally maintain the spatial context required for long-horizon navigation. To this end, we study goal-initialized navigation, where the goal is provided only once at the beginning of an episode, with no subsequent external relative-goal updates. We present GUIDE, an end-to-end RL framework that jointly learns navigation and internal directional awareness from onboard observations. GUIDE leverages multi-frequency proprioceptive history to capture egomotion and auxiliary spatial-anchor prediction to maintain task-relevant spatial states, while temporal depth observations provide local environmental geometry. The navigation policy is trained entirely in simulation and transferred zero-shot to the real world. Across cluttered environments and structured mazes, GUIDE reliably avoids obstacles, escapes dead ends, and reaches distant goals using only onboard sensing. These results demonstrate that robust sim-to-real legged navigation can be achieved without continuously providing external robot-to-goal estimation, opening a promising direction for future research on more self-contained end-to-end navigation. Video demonstrations are available at: `https://guide-navigation.github.io/`.**

I. Introduction

Legged robots possess inherent agility to traverse complex terrains and navigate in confined environments [1]–[4]. Such environments often contain narrow passages, occlusions, unexplored regions, and dead ends, where onboard sensing provides only partial observations of the surroundings. Robust navigation therefore requires not only reactive obstacle avoidance, but also the ability to maintain spatial context over extended horizons and avoid myopic decisions that may lead to locally attractive but globally unproductive states [5], [6].

End-to-end learning provides a promising approach to this problem by directly mapping onboard observations to navigation actions [7]–[10]. Recent reinforcement learning (RL) methods have demonstrated increasingly capable obstacle avoidance and long-range navigation behaviors [5], [11], [12]. However, despite learning the navigation policy end-to-end, these approaches commonly rely on continuously updated robot-to-goal states provided by external localization or state estimation modules. Consequently, an important part of the navigation problem—maintaining the robot's evolving spatial relation to the goal—remains outside the learned policy. This raises a natural question when pushing the limits of end-to-end sim-to-real RL navigation: *can a legged robot navigate over long horizons without continuous external robot-to-goal updates by maintaining the required spatial context internally?*

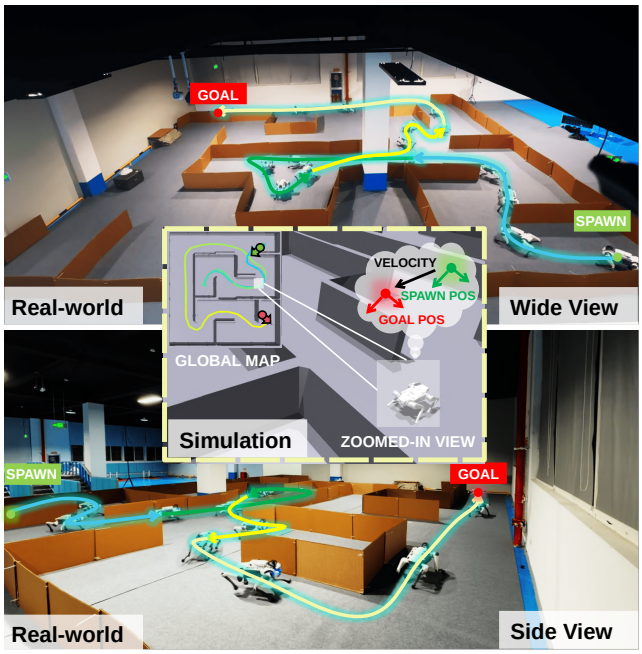


Fig. 1. GUIDE cultivates internal egomotion and directional awareness for end-to-end quadruped navigation, allowing the policy to traverse complex maze-like terrains and escape dead ends with the target location given only at initialization. The central simulation inset illustrates joint learning of navigation and spatial awareness. The top and bottom panels show wide and side views of a real-world deployment, where the gradient-colored curve traces the robot's trajectory through multiple dead ends to the goal.

To investigate this question, we study **goal-initialized navigation**, where the goal is provided only once at the beginning of an episode and no subsequent external relative-goal updates are available to the policy. This setting makes long-horizon navigation particularly challenging under partial observability. The robot must continuously reason about how its own motion changes its relation to the initial goal, while simultaneously using local observations to avoid obstacles, recognize unproductive routes, and escape dead ends. Importantly, legged robots naturally provide rich proprioceptive histories through their joint encoders and inertial sensors, which contain the motion information of the robot [13], [14]. This motivates us to investigate whether task-relevant

*Equal contribution. †Corresponding authors.
[1]Institute of Cyber-Systems and Control, Zhejiang University; [2]Institute of Artificial Intelligence (TeleAI), China Telecom; [3]University of Oxford; [4]Center for Robotics, University of Bonn; [5]The Hong Kong University of Science and Technology (Guangzhou).

egomotion and directional context can be learned directly from proprioception along with the navigation policy rather than continuously supplied by an external estimator.

To this end, we propose GUIDE, an end-to-end RL framework that jointly learns navigation and internal directional awareness from onboard observations. GUIDE processes multi-frequency proprioceptive history to capture both long-term motion context and fine-grained egomotion, while temporal depth observations encode the geometry information of the environment. During training, an auxiliary spatial-anchor predictor supervises the proprioceptive representation to estimate task-relevant spatial states, including the relative goal location, relative spawn position, and body velocity in the ego-centric frame. These learned representations are integrated with depth observations to support long-horizon navigation without external streaming relative-goal inputs. The navigation policy is trained entirely in simulation and transferred zero-shot to a real quadruped robot. Through extensive simulation and real-world experiments, we show that GUIDE can navigate dense clutter, escape structural dead ends, and traverse challenging maze environments using only onboard observations after goal initialization, as shown in Fig. 1. The main contributions of this work are summarized as follows:

- We pursue the limit of sim-to-real RL based end-to-end navigation. We study goal-initialized navigation for quadruped robots, where the goal is provided only at initialization, and no external streaming relative-goal updates are available during navigation.
- We propose GUIDE, an end-to-end RL framework that combines temporal depth perception with multi-frequency proprioceptive history and auxiliary spatial-anchor prediction, enabling the policy to learn task-relevant egomotion and directional context internally.
- We validate GUIDE extensively in simulation and demonstrate zero-shot transfer to real-world environments, showing robust goal reaching and dead-end escaping using only onboard sensing.

II. Related Work

A. Learning-Based Local Navigation

Recent works in imitation learning leverage expert demonstrations or large-scale offline datasets to map observations directly to control commands [15]–[18], which allow robots to inherit expert-level decision-making but are typically confined to the specific data distributions. Imperative learning paradigms [19], [20] address local navigation by optimizing differentiable objectives to generate collision-free and semantically-informed paths from onboard perception. Complementary to these, reinforcement learning (RL) is widely adopted to discover agile obstacle avoidance and terrain traversal strategies through large-scale interaction with simulated environments [21]–[25]. Despite their proficiency in local maneuvers, these methods may still make myopic decisions and become trapped in dead ends under partial observability [26]–[28].

B. Long-horizon Mapless Navigation

Recent research has increasingly focused on enabling legged robots to overcome the limitations of short-horizon navigation and escape local minima without prior-map information directly fed into the policy. Lee et al. [11] develop a hierarchical RL system for wheeled-legged robots that can locally detour dead ends by feeding historically visited positions into the policy. Yang et al. [5] introduce a spatially enhanced recurrent network, demonstrating superior spatial memorization in maze-like environments with the current relative goal position provided to the policy at every step. REASAN [12] utilizes a modular LiDAR-based framework, comprising locomotion, safety-shield, and exteroceptive estimation to facilitate effective detouring and dead-end escape in dynamic environments. Although these methods demonstrate the capability of RL to handle long-horizon navigation, they remain dependent on external state estimation for continuous relative-goal updates in real-world deployment. This dependency leaves a key part of navigation outside the learned policy, namely tracking the robot's evolving spatial relation to the goal.

C. Proprioceptive Odometry

Proprioceptive odometry provides essential ego-motion estimates for legged robot navigation in unmapped or visually degraded environments. Classical filtering methods (e.g., Extended Kalman Filters [29]) rely on idealized contact assumptions, making quadrupedal locomotion prone to cumulative drift from ground slippage and unmodeled terrain dynamics [30], [31]. To address these constraints, learning-based approaches directly map raw IMU and joint sequences to motion estimates, capturing complex contact behavior without explicit kinematic modeling [32], [33]. Legolas [13] learns to predict relative pose increments from proprioceptive histories entirely in simulation, achieving zero-shot transfer to real quadrupeds. AutoOdom [14] combines simulation pretraining with autoregressive fine-tuning on real-world data to improve odometry robustness. These results suggest that proprioceptive histories contain sufficient motion cues to infer robot egomotion and support long-horizon spatial awareness. Building on this insight, we investigate whether a navigation policy can learn to maintain this spatial context internally. While these odometry methods accumulate predicted motion increments to recover robot pose, GUIDE jointly learns direct prediction of task-relevant spatial anchors and navigation within a single-stage end-to-end framework, bringing relative-goal tracking into the learned policy without continuous external goal updates.

III. Method

A. Goal-Initialized Navigation Formulation

We formulate the navigation task as a Partially Observable Markov Decision Process (POMDP) [34]. At each time step t, the robot cannot access the global state and must make navigation decisions based on its onboard proprioception and depth inputs. In our goal-initialized setting, the robot is placed at a collision-free start position at the beginning

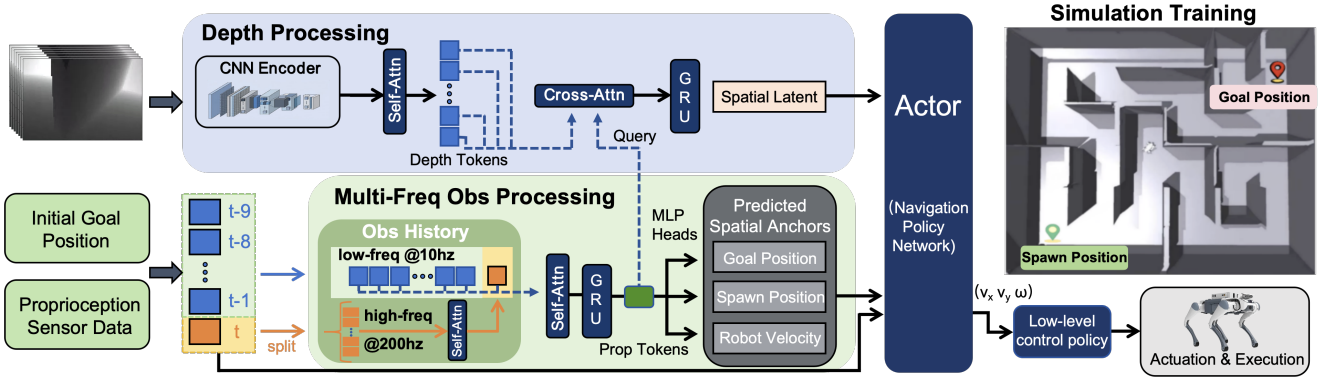


Fig. 2. **Overview of the GUIDE framework.** Multi-frequency proprioceptive history is processed into proprioceptive tokens, which are supervised to predict spatial anchor vectors to cultivate egomotion and directional awareness. Concurrently, depth buffers are encoded and fused with these tokens via cross-attention to yield spatial latents. Finally, the Actor aggregates these representations with the latest proprioceptive state to produce the twist commands. All navigation modules are jointly optimized end-to-end from scratch.

of each episode ($t = 0$) and receives an initial relative target $g^0 = [x_g^0, y_g^0, \psi_g^0]^T \in \mathbb{R}^3$ expressed in its egocentric frame. Crucially, no streaming goal updates are provided in the rest of the episode ($t > 0$). To address the absence of continuous external state estimation, we introduce a spatial anchor prediction mechanism (detailed in Section III-B) that is jointly trained with the navigation policy to estimate the spatial context relevant to the task $\hat{c}_t$ from the proprioceptive history. Consequently, the policy generates actions conditioned on current observation o_t, observation history $\mathcal{O}_t$, and the predicted spatial context $\hat{c}_t$, *i.e.*, $a_t \sim \pi_\theta(\cdot \mid o_t, \mathcal{O}_t, \hat{c}_t)$. The optimization objective is to learn a policy π_θ that maximizes the expected discounted return over the initial goal distribution $p(g^0)$ and the transition dynamics of the environment: $J(\pi_\theta) = \mathbb{E}_\tau\big[\sum_{t=0}^{T} \gamma^t r(s_t, a_t, g^0)\big]$, where τ denotes the induced trajectories by the policy.

B. GUIDE Framework

We adopt an asymmetric actor-critic architecture optimized via Proximal Policy Optimization (PPO) [35]. GUIDE follows a hierarchical control structure: the high-level navigation policy operates at 10 Hz to reason over multi-modal observations and produce twist commands, while a low-level locomotion controller runs at 50 Hz to track these commands. An overview of the framework is illustrated in Fig. 2.

The navigation policy takes three types of input: raw proprioceptive sensor measurements, raw depth images, and the initial target g^0 expressed in the robot's initial egocentric frame. The proprioceptive observations are obtained from joint encoders and the IMU. Each proprioceptive observation is a 46-dimensional vector:

$$p_t = [t_{\text{ep}}, a_{t-1}, \omega_t, g_t^{\text{grav}}, \theta_t, \dot{\theta}_t, \theta_{t-1}^{\text{cmd}}]^T, \tag{1}$$

where $t_{\text{ep}} \in \mathbb{R}$ denotes the elapsed time since episode initialization, and $a_{t-1} \in \mathbb{R}^3$ is the previously executed high-level twist command. $\omega_t \in \mathbb{R}^3$ is the base angular velocity, and $g_t^{\text{grav}} \in \mathbb{R}^3$ is the projected gravity vector in the base frame. $\theta_t \in \mathbb{R}^{12}$ and $\dot{\theta}_t \in \mathbb{R}^{12}$ denote the current joint positions and joint velocities obtained from the motor encoders, respectively. Finally, $\theta_{t-1}^{\text{cmd}} \in \mathbb{R}^{12}$ denotes the previous low-level joint position targets. The depth input is structured as an eight-frame history buffer $\mathcal{D}_t = \{D_{t-7}, \ldots, D_t\}$, where each individual depth frame is down-sampled: $D_t \in \mathbb{R}^{30\times 43}$.

Multi-Frequency Proprioceptive Processing The proprioceptive inputs are encoded through two complementary temporal streams: a low-frequency history stream $\mathcal{P}_t^{\text{l}} = \{p_{t-N+1}^{\text{l}}, \ldots, p_t^{\text{l}}\}$ comprising $N = 10$ samples collected at the policy rate of 10 Hz, and a high-frequency recent stream $\mathcal{P}_t^{\text{h}} = \{p_{t-M+1}^{\text{h}}, \ldots, p_t^{\text{h}}\}$ sampled at 200 Hz between the previous and current policy steps, yielding $M = 20$ samples per policy interval (notably $p_t^{\text{h}} \equiv p_t^{\text{l}}$ at current step t). The former tracks the historical motion context over a relatively longer temporal horizon, while the latter captures fine-grained transient dynamics. The high-frequency observations are first encoded frame-wise, augmented with positional encodings, and processed by self-attention. We retain the last output token as the recent-motion embedding. This enriched token replaces the latest low-frequency token and is concatenated with the preceding $N-1$ encoded low-frequency tokens to form a unified sequence. The sequence is again augmented with positional encodings and processed by a second self-attention layer. Its last output token, integrating information from both temporal streams, is then fed into a GRU network, which maintains memory across policy steps and outputs the proprioception token z_t^{p}.

Spatial Anchor Prediction To guide the policy to extract and utilize the proprioception token, we add an auxiliary supervision signal through a spatial anchor predictor to the token z_t^{p}. Unlike the typical odometry-based pipeline that integrates incremental motion estimates [13], [14] and derives task states from the resulting pose, this predictor employs three lightweight MLP heads to directly estimate the robot's body-frame velocity $\hat{v}_t$, relative goal position $\hat{g}_t$, and relative spawn position $\hat{s}_t$. During training, these predictions are supervised using privileged simulation states, where the estimation loss is defined as:

$$\mathcal{L}_{\text{est}} = \|\hat{v}_t - v_t\|_2^2 + \mathcal{H}_\delta(\hat{g}_t - g_t) + \mathcal{H}_\delta(\hat{s}_t - s_t), \tag{2}$$

Here, v_t, g_t, and s_t denote the corresponding ground-truth simulation states, and $\mathcal{H}_\delta(\cdot)$ denotes the smooth L1 (Huber) loss, with $\delta = 1$ for both position terms.

Depth Processing Each individual frame within $\mathcal{D}_t$ is independently processed by a shared convolutional encoder

TABLE I: COMPLETE REWARD TERMS AND COEFFICIENTS.

Reward Term	Formulation	Coeff.
Sparse task rewards		
Position reaching	$\mathbb{I}_{\text{complete}}/(1+(e_t^{\text{pos}}/\sigma_{\text{pos}})^2)$	3.0
Heading reaching	$\mathbb{I}_{\text{complete}}/(1+(e_t^{\text{head}}/\sigma_{\text{head}})^2)$	1.5
Bad termination	$\mathbb{I}_{\text{bad}}$	-1.0
Dense shaping rewards		
Euclidean progress	$\frac{\max(0, d_{\text{euc}}^{\text{best}} - d_t^{\text{euc}})}{d_{t=0}^{\text{euc}}}$	60.0
Maze-cell progress	$\frac{\max(0, d_{\text{cell}}^{\text{best}} - d_t^{\text{cell}})}{d_{t=0}^{\text{cell}}}$	60.0
Near-goal velocity damping	$\exp(-\sum_i \lvert a_{t,i}\rvert / a_i^{\max})\mathbb{I}_{\text{soft}}$	5.0
No-motion penalty	$\mathbb{I}_{\text{static}}$	-5.0
Collision penalty	$\mathbb{I}_{\text{coll}}$	-50.0
Command overspeed penalty	$\sum_i \max(\lvert a_{t,i}^{\text{raw}}\rvert - \lambda a_i^{\max}, 0)^2$	-2.5
Command rate	$\lVert a_t - a_{t-1}\rVert_2^2$	-0.05
Joint acceleration	$\lVert \ddot{\theta}_t\rVert_2^2$	-2.5×10^{-7}
Joint torque	$\lVert \tau_t\rVert_2^2$	-2.0×10^{-4}
Linear velocity (z)	$v_{t,z}^2$	-0.1
Roll-pitch angular velocity	$\lVert \omega_{t,xy}\rVert_2^2$	-0.005
Orientation	$\lVert g_{t,xy}^{\text{grav}}\rVert_2^2$	-0.1

Parameters: $\sigma_{\text{pos}} = 0.2\,\text{m}$, $\sigma_{\text{head}} = 0.5\,\text{rad}$, $\sigma_{\text{soft}} = 2.0\,\text{m}$, $\lambda = 1.3$.

to extract depth features. These frame-wise features are then integrated through a self-attention layer to fuse the temporal information across the buffer, producing the depth tokens z_t^{d}. To condition the depth features on the robot's motion state, a cross-attention module is employed using the proprioceptive token z_t^{p} as the query and the depth tokens z_t^{d} as keys and values. The resulting cross-modal features are passed through a GRU layer to produce the final spatial latent feature z_t^{s}.

Ultimately, the navigation policy network takes the concatenated vector $[z_t^{\text{s}}, \hat{v}_t, \hat{g}_t, \hat{s}_t, p_t^{\text{l}}]$ as input and outputs the egocentric twist commands $a_t = [v_x, v_y, \omega_z]^T$, where p_t^{l} denotes the latest frame of proprioceptive observations. The commands are subsequently executed on the robot by the low-level locomotion controller, which is trained following the framework established by Long et al. [36].

C. Reward Design and Multi-Critic Value Estimation

Reward Function Design Our reward design combines sparse task-level rewards with dense progress rewards. The complete list of reward terms is provided in Table I. Task-level rewards (r_{pos}, r_{head}) follow the formulation of r_{track}, while dense progress rewards ($r_{\text{prog}}^{\text{euc}}$, $r_{\text{prog}}^{\text{cell}}$) follow the formulation of r_{prog}:

$$r_{\text{track}} = \frac{\mathbb{I}_{\text{complete}}}{1+(e/\sigma)^2}, \qquad r_{\text{prog}} = \frac{\max(0, d^{\text{best}} - d_t)}{d_{t=0}}, \tag{3}$$

where e and σ denote the corresponding tracking error and scale, and $\mathbb{I}_{\text{complete}}$ activates only when the relative goal distance d_t falls below σ_{pos}. $\mathbb{I}_{\text{bad}}$ indicates failure termination when the robot falls over. $\mathbb{I}_{\text{soft}}$ activates when the goal is within σ_{soft} and no obstacle lies between the robot and the goal. For progress terms, d_t and d^{best} denote the current and previous minimum distances, and progress is normalized by the initial distance $d_{t=0}$. Cluttered environments use the Euclidean progress reward $r_{\text{prog}}^{\text{euc}}$, with d denoting the straight-line distance to the goal. Maze environments use $r_{\text{prog}}^{\text{cell}}$, with d denoting the maze-graph connectivity distance to the goal.

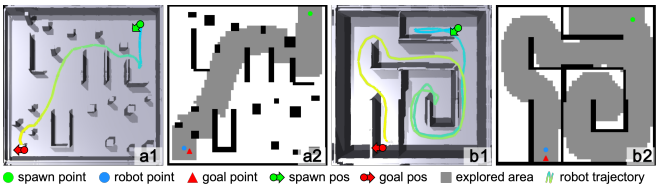


Fig. 3. **(a1, b1):** Cluttered and maze terrains, with curves in gradient color indicating the robot's trajectories. **(a2, b2):** Overlaid visualizations of the occupancy and explored-region maps, where black denotes obstacles, gray denotes explored free space, and white denotes unexplored free space. The two maps are stacked as separate input channels for the privileged critic.

Privileged Critic Learning Under the asymmetric architecture, the value networks leverage privileged information from simulation to improve value estimation and guide policy optimization. As shown in Fig. 3, we provide the critic with a downsampled top-down occupancy map $\mathcal{M}_{\text{occ}}$ and an explored-region map $\mathcal{M}_{\text{exp}}$, thereby providing the critic with explicit global information about the environmental geometry and the robot's traversal trajectory. These maps are stacked as a two-channel tensor and processed by a dedicated convolutional network. **Multi-Critic Paradigm:** To better estimate the mixture of dense and sparse rewards with distinct temporal characteristics, we introduce a Multi-Critic (MuC) architecture [37], [38]. The total reward is partitioned into a sparse task group r_t^{s} and a dense shaping group r_t^{d}. The MuC module outputs independent value estimates $V_\phi^k(s_t)$ for $k \in \{\text{s}, \text{d}\}$, optimized against their bootstrapped returns $\hat{R}_t^k$ by minimizing $\mathcal{L}_{\text{value}} = \sum_{k\in\{\text{s,d}\}} \mathbb{E}[(\hat{R}_t^k - V_\phi^k(s_t))^2]$. We then compute the unified normalized advantage $\hat{A}_t$ from the individual generalized advantage estimates (GAE) [39] $\hat{A}_t^k$:

$$\hat{A}_t = \Big(\sum_{k\in\{\text{s,d}\}} w_k \hat{A}_t^k - \mu_{\text{MuC}}\Big)/\sigma_{\text{MuC}}, \tag{4}$$

where w_k denotes the reward-group weight, while μ_{MuC} and σ_{MuC} are the batch mean and standard deviation of the combined advantages.

D. Training Details

Terrain Design. To train a policy with both reactive obstacle avoidance and long-horizon reasoning capabilities, we procedurally generate two distinct terrain types. As illustrated in Fig. 3, **Cluttered terrains** are constructed by randomly placing specific obstacle categories, including narrow alleys, L-shaped corners, and pillars. The resulting dense and irregular geometry creates environments with frequent occlusions and local concave traps. **Maze terrains** are grid-based perfect mazes generated by randomized depth-first search (DFS) [40]. These loop-free structures guarantee a unique path between any two grid cells while providing scalable topological difficulty through branching corridors and dead ends, requiring the robot to retrace its route after entering an incorrect branch.

Training Curriculum. We adopt a curriculum that progressively increases both terrain complexity and initial robot-

to-goal distance. Specifically, terrain complexity is increased by raising obstacle density in cluttered environments and increasing the number of grid cells in mazes. The initial robot-to-goal distance is progressively increased, measured by Euclidean distance in cluttered environments and maze-graph connectivity distance in mazes. More precisely, the latter counts the transitions between adjacent cells along the unique traversable path from the start cell to the goal cell.

Simulation Setup. We train the 12-DoF Deep-Robotics Lite3 robot using Isaac Gym across 4,096 parallel environments on a single NVIDIA H100 GPU. During training, we randomize dynamics parameters and inject noise into proprioceptive sensor measurements and depth images. The entire navigation reinforcement learning process is completed end-to-end in a single unified phase without any pretraining, with the policy typically converging within 20,000 iterations.

IV. Experimental Results

A. Simulation Benchmarks and Ablations

To assess the contributions of GUIDE's components and compare against baseline methods, we consider the following configurations. First, for internal component ablations: (i) **w/o High-Frequency Prop** removes the 200 Hz high-frequency proprioceptive observation branch, restricting the policy to utilize only the 10 Hz observation history processed via self-attention; (ii) **w/o Target Prediction**, (iii) **w/o Spawn Prediction**, and (iv) **w/o Velocity Prediction** deactivate their respective prediction heads; (v) **w/o Multi-Critic** replaces the multi-critic architecture with a standard single-critic network to predict the total return. Second, external baselines: (vi) **GUIDE-Legolas** replaces GUIDE's spatial-anchor predictor with the pretrained Legolas odometry model [13], deriving body-frame velocity from its predicted relative pose increments and accumulating these increments to compute the current relative goal and spawn positions; (vii) **SRU** [5], which is trained with continuous streaming ground-truth relative-goal updates. Finally, to investigate performance boundaries, we evaluate two ground-truth oracle variants: (viii) **GUIDE-GT-Inference**, which directly replaces the predicted goal and spawn positions with ground-truth values from the simulation in test experiments. (ix) **GUIDE-GT-Train**, which completely removes the spatial-anchor predictor and is trained from scratch, receiving ground-truth relative goal and spawn positions at every control step during both training and inference.

We evaluate the compared policies across cluttered and maze terrains at two difficulty levels, resulting in four benchmark settings shown in Fig. 4. Except for SRU and the oracle variants, all methods receive the goal only at initialization. Performance is quantified by three key metrics: (i) **Success Rate (SR)**, computed as the percentage of trials reaching within a 0.2m radius of the goal; (ii) **Arrival Time (AT)**, defined as the mean episode duration across all trials, subject to a 60 s timeout; and (iii) **Collision Rate (CR)**, defined as the percentage of episodes with at least one physical collision. For each benchmark, we perform 10 evaluation runs, each comprising 1,000 episodes across 50 layouts

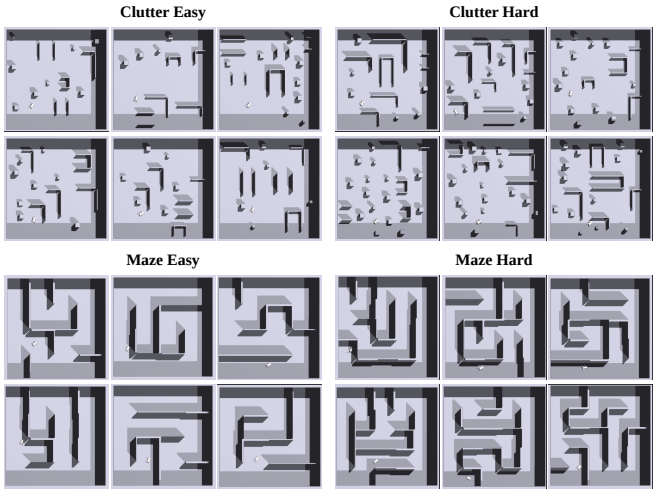


Fig. 4. **Example simulation benchmark layouts.** Benchmark difficulty is determined by obstacle density in cluttered terrains and the number of grid cells in maze terrains. Each evaluation run uses 50 distinct layouts per setting, generated using randomized seeds.

generated using a different set of random seeds. Table II reports the mean and standard deviation across these runs.

Beyond task performance, we further evaluate the spatial anchor prediction accuracy to understand how the supervised predictor contributes to the ablation results. Specifically, this analysis examines the components that shape the spatial anchor representation, including high-frequency proprioceptive input and individual prediction heads, while also comparing GUIDE's spatial estimation accuracy with Legolas [13]. Under the same benchmark settings, we measure the mean tracking error (MTE) of the relative target position, the relative spawn position, and the body velocity, averaged across all policy steps within the evaluation episodes. The corresponding results and standard deviation are shown in Table III. Based on these quantitative results, we derive the following key insights.

High-Frequency Proprioception is Crucial for Egomotion and Directional Awareness. The **w/o High-Frequency Prop** variant suffers a severe performance collapse across all tasks, almost completely failing to reach the target within the time limit. Instead of actively navigating, the robot quickly becomes trapped in local dead ends, which explains its low collision rate in the maze environment. As revealed in Table III, the policy entirely loses its internal tracking of the target and spawn positions, alongside degraded velocity estimation in cluttered environments, which suggests that maintaining a reliable sense of egomotion and direction under partial observability strongly relies on the fine-grained motion information provided by high-frequency proprioceptive observations. This capability is particularly important in our goal-initialized setting, where the navigation policy must learn to maintain spatial awareness internally without continuous external relative-goal updates.

Spatial Anchor Predictions Jointly Shape Spatial Context. GUIDE consistently outperforms all variants lacking individual prediction heads. Notably, **w/o Target Prediction** suffers a larger decline in task performance than **w/o Spawn**

TABLE II: QUANTITATIVE COMPARISON ACROSS FOUR SIMULATION BENCHMARK SETTINGS.

Policy Variant	Cluttered Environments						Maze Environments					
	Easy			Hard			Easy			Hard		
	SR↑ [%]	AT↓ [s]	CR↓ [%]	SR↑ [%]	AT↓ [s]	CR↓ [%]	SR↑ [%]	AT↓ [s]	CR↓ [%]	SR↑ [%]	AT↓ [s]	CR↓ [%]
GUIDE (Full)	99.9±0.1	6.85±0.8	3.20±0.6	99.9±0.1	7.44±1.0	4.40±0.8	99.9±0.1	15.7±0.9	3.40±0.3	99.6±0.3	22.1±0.9	4.80±0.4
(i) w/o High-Freq. Prop	0.16±0.2	57.9±1.8	12.8±1.7	0.08±0.1	58.6±1.2	13.3±3.1	0.06±0.1	59.2±0.6	2.80±0.4	0.01±0.0	59.8±0.1	3.10±0.4
(ii) w/o Target Prediction	97.8±0.4	8.22±0.9	6.40±1.2	96.7±0.6	8.86±0.9	8.20±1.5	98.2±0.4	18.2±1.3	6.20±0.6	70.9±3.6	34.9±2.4	8.40±0.9
(iii) w/o Spawn Prediction	98.8±0.6	7.50±1.1	4.40±0.9	98.2±0.5	7.66±1.1	4.80±1.4	98.9±0.4	16.5±1.5	4.90±0.7	90.1±2.1	26.7±2.8	6.60±1.3
(iv) w/o Velocity Prediction	99.0±0.3	8.84±1.7	7.40±2.1	98.5±0.5	9.50±2.0	12.2±4.3	96.3±2.5	19.9±2.4	7.80±1.9	53.5±7.2	39.4±4.5	11.30±3.6
(v) w/o Multi-Critic	95.2±2.1	10.6±1.4	13.3±4.3	88.5±2.7	11.8±2.1	15.4±5.5	82.0±3.4	26.9±2.9	6.50±2.3	49.1±6.8	41.0±4.3	5.40±2.1
(vi) GUIDE-Legolas [13]	82.5±3.1	17.6±2.1	5.0±1.2	77.6±4.3	20.6±3.2	6.3±1.6	76.9±3.5	28.5±2.3	5.50±1.3	63.3±5.4	39.8±3.0	6.40±1.6
(vii) SRU [5]	96.2±0.7	10.3±0.5	8.04±2.1	93.6±1.0	12.8±0.7	9.65±4.0	85.3±2.8	24.2±1.2	5.24±0.9	76.3±4.0	41.6±1.7	5.80±1.2
(viii) GUIDE-GT-Inference	100.0±0.0	6.47±0.7	2.60±0.6	99.9±0.1	6.54±0.9	3.10±0.8	99.9±0.1	15.2±0.9	2.40±0.2	99.9±0.1	20.7±1.0	3.60±0.5
(ix) GUIDE-GT-Train	100.0±0.0	6.43±0.7	3.55±0.6	100.0±0.0	6.69±0.8	5.62±0.9	99.9±0.1	14.8±0.3	4.24±0.5	99.8±0.1	20.6±0.7	6.02±0.7

TABLE III: SPATIAL ANCHOR PREDICTION ERROR ACROSS FOUR BENCHMARK SETTINGS IN SIMULATION.

Policy Variant	Cluttered Environments						Maze Environments					
	Easy			Hard			Easy			Hard		
	Tgt. [m]	Spn. [m]	Vel. [m/s]	Tgt. [m]	Spn. [m]	Vel. [m/s]	Tgt. [m]	Spn. [m]	Vel. [m/s]	Tgt. [m]	Spn. [m]	Vel. [m/s]
GUIDE (Full)	0.21±0.09	0.17±0.09	0.12±0.07	0.23±0.10	0.18±0.09	0.12±0.07	0.22±0.12	0.18±0.12	0.11±0.07	0.31±0.17	0.27±0.15	0.11±0.08
w/o High-Freq. Prop	3.43±2.72	2.92±2.15	0.18±0.12	3.37±2.85	2.84±2.26	0.18±0.13	3.10±2.34	2.82±2.14	0.10±0.09	3.16±2.55	2.76±2.34	0.11±0.09
w/o Target Prediction	–	0.13±0.07	0.11±0.07	–	0.17±0.08	0.13±0.09	–	0.32±0.17	0.11±0.07	–	0.54±0.35	0.10±0.07
w/o Spawn Prediction	0.34±0.15	–	0.12±0.08	0.33±0.18	–	0.12±0.08	0.38±0.20	–	0.11±0.07	0.46±0.25	–	0.11±0.08
w/o Velocity Prediction	0.29±0.14	0.18±0.12	–	0.29±0.17	0.19±0.15	–	0.37±0.21	0.33±0.20	–	0.67±0.37	0.65±0.44	–
GUIDE-Legolas [13]	0.48±0.36	0.51±0.66	0.07±0.04	0.51±0.43	0.53±0.67	0.07±0.04	0.40±0.28	0.33±0.28	0.07±0.03	0.44±0.30	0.42±0.35	0.07±0.03

Prediction, as shown in Table II. We attribute this gap to the more direct goal-oriented guidance provided by the target position, while the spawn prediction requires the policy to implicitly infer the spatial relation between the start and the target. Among the three prediction-head ablations, **w/o Velocity Prediction** experiences the most severe degradation in both success rate and collision avoidance. These results indicate that robust navigation requires both instantaneous motion cues from velocity prediction and long-horizon positional anchors from target and spawn prediction, which together help the policy construct a coherent spatial context.

GUIDE Effectively Exploits Learned Spatial Context for Robust Navigation. GUIDE achieves higher SR than SRU [5] across all four benchmarks. In observed SRU failures, the policy repeatedly cycles between neighboring dead ends, whereas GUIDE backtracks and escapes. GUIDE-Legolas yields lower instantaneous velocity error. However, accumulating its frame-to-frame pose estimates to recover the robot pose and transform the goal and spawn positions into the current robot frame results in larger position errors and lower SR. This highlights the advantage of our proposed method that directly learns task-relevant spatial anchors jointly with the navigation policy. Without prediction errors in navigation process, both oracle variants (**GUIDE-GT-Inference** and **GUIDE-GT-Train**) achieve slightly better performance than **GUIDE** in terms of success rate and arrival time. Although this improvement reveals the remaining limitation of the learned internal navigation states, the small gap also indicates that GUIDE effectively learns and exploits such spatial context from partial observations to accomplish long-horizon navigation. This supports the central hypothesis of GUIDE: an end-to-end RL policy can maintain internal egomotion and directional awareness and successfully navigate complex environments when the target is provided only at initialization. Additionally, the significant performance drop in **w/o Multi-Critic** demonstrates that decoupling the value estimation for dense and sparse rewards is critical for policy learning.

B. Real-World Experiments

We deploy our framework on a Deep-Robotics Lite3 quadruped robot. The policy runs onboard on an NVIDIA Jetson Orin NX, using proprioceptive measurements from the robot and depth images from an Intel RealSense D435i camera. The depth stream is processed with the official RealSense filtering pipeline, updating depth frames at 10 Hz. For real-world quantitative evaluation, we use a Livox MID360 LiDAR with Faster-LIO [41] for mapping and robot pose estimation. This LiDAR-based pipeline is used solely for evaluation and provides no input to the navigation policy.

In-Lab Evaluations: We evaluate GUIDE in cluttered and maze environments, each with Easy and Hard settings corresponding to those used in simulation. Representative deployments are shown in Fig. 5. The longest executed trajectories reach $24.5\,\mathrm{m}$ and $56.1\,\mathrm{m}$ in the Easy and Hard settings, respectively. During physical deployment, the robot is programmed to stop autonomously when its internally estimated distance to the target falls below $0.2\,\mathrm{m}$. To quantify



Fig. 5. **Real-world deployments.** GUIDE navigates through **(a)** maze and **(b)** cluttered environments. **(c)** illustrates escape from multiple dead ends during a representative navigation trial, with **(d)** providing an overhead view of the same trial. **(e, f)** show navigation under different start–goal configurations. Additional deployments include **(g, h)** narrow office corridors and outdoor grasslands with **(i)** dense vegetation and **(j)** dynamic obstacles. Colored curves in (a–f) indicate executed trajectories, while top-left insets in (g–j) show onboard depth observations.

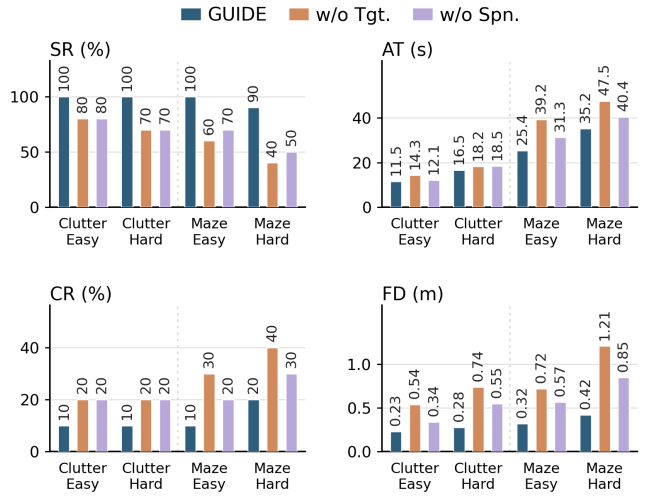


Fig. 6. **Quantitative results of real-world environments. GUIDE** achieves the best results across all metrics. The robot autonomously stops when its internally estimated distance to the goal falls below 0.2 m. Therefore, the resulting average final distances **(FDs)** of approximately 0.2–0.3 m indicate that the learned internal goal representation remains reasonably accurate during real-world deployment, supporting the zero-shot sim-to-real transfer of the policy.

the accuracy of the learned internal directional awareness, we introduce an additional metric: **Final Distance (FD)**, defined as the actual physical distance between the robot's base and the goal location when the robot "thinks" it has reached the goal based on its internal direction awareness. For physical deployment, we evaluate GUIDE and the two ablation variants with the highest simulation success rates. Each method is evaluated over 10 independent physical trials per setting, with the quantitative results detailed in Fig. 6. Across all scenarios, GUIDE achieves the best overall performance and reaches the target with the highest precision, demonstrating the robustness of our framework.

In-the-wild Deployments: We further deploy GUIDE in unstructured environments unseen during training. As illustrated in Fig. 5, these scenarios range from long office corridors to outdoor grasslands, where the robot must dynamically avoid obstacles and detour around dense vegetation. Despite drastic changes in environment appearance, physical terrain attributes, lighting, and obstacle shapes, GUIDE maintains stable navigation behavior using solely raw onboard sensing, achieving successful zero-shot sim-to-real transfer and overcoming real-world domain shifts.

V. Conclusion

In this work, we presented GUIDE, an end-to-end reinforcement learning framework for goal-initialized legged navigation. By utilizing multi-frequency raw proprioception and raw depth information, GUIDE enables legged robots to cultivate internal egomotion and directional awareness, allowing the navigation policy to maintain task-relevant spatial context internally. Extensive evaluations in both simulated and real-world environments demonstrate that our approach empowers the robot to robustly traverse clutters and escape maze dead-ends without the reliance on continuous external goal references, validating the effectiveness of the learned spatial awareness alongside zero-shot sim-to-real transfer ability.

However, we observe that GUIDE occasionally misses traversable openings in confined spaces, potentially due to the limited field of view of the onboard depth sensor. Future work will explore active perception and additional sensor modalities to expand the robot's local spatial observations. Additionally, we will investigate how to extend GUIDE to larger-scale navigation scenarios while maintaining reliable internal spatial awareness.

REFERENCES

[1] D. Hoeller, N. Rudin, D. Sako, and M. Hutter, "Anymal parkour: Learning agile navigation for quadrupedal robots," *Science Robotics*, vol. 9, no. 88, p. eadi7566, 2024.

[2] X. Cheng, K. Shi, A. Agarwal, and D. Pathak, "Extreme parkour with legged robots," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 11 443–11 450.

[3] Y. Wu, J. Kuang, S. Khorshidi, X. Niu, L. Klingbeil, M. Bennewitz, and H. Kuhlmann, "Doglegs: Robust proprioceptive state estimation for legged robots using multiple leg-mounted imus," in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, pp. 8856–8863.

[4] G. Caroleo, T. Mahamoodally, M. Manzardo, J. Jin, M. Pontin, M. Mattamala, R. Vidoni, P. Maiolino, and M. Fallon, "Quadruped obstacle avoidance and footstep planning with distributed low-cost time-of-flight sensors," *IEEE Robotics and Automation Letters*, pp. 1–8, 2026.

[5] F. Yang, P. Frivik, D. Hoeller, C. Wang, C. Cadena, and M. Hutter, "Spatially-enhanced recurrent memory for long-range mapless navigation via end-to-end reinforcement learning," *The International Journal of Robotics Research*, p. 02783649251401926, 2025.

[6] J. Jeong, M. Yoon, S. Choi, H. Shin, T. Yang, and S.-e. Yoon, "Hipan: Hierarchical posture-adaptive navigation for quadruped robots in unstructured 3d environments," *arXiv preprint arXiv:2604.26504*, 2026.

[7] L. Tai, G. Paolo, and M. Liu, "Virtual-to-real deep reinforcement learning: Continuous control of mobile robots for mapless navigation," in *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*. IEEE, 2017, pp. 31–36.

[8] E. Wijmans, A. Kadian, A. S. Morcos, S. Lee, I. Essa, D. Parikh, M. Savva, and D. Batra, "Dd-ppo: Learning near-perfect pointgoal navigators from 2.5 billion frames," in *International Conference on Learning Representations*, 2019.

[9] H. Shi, L. Shi, M. Xu, and K.-S. Hwang, "End-to-end navigation strategy with deep reinforcement learning for mobile robots," *IEEE Transactions on Industrial Informatics*, vol. 16, no. 4, pp. 2393–2402, 2019.

[10] W. Huang, Y. Zhou, X. He, and C. Lv, "Goal-guided transformer-enabled reinforcement learning for efficient autonomous navigation," *IEEE Transactions on Intelligent Transportation Systems*, vol. 25, no. 2, pp. 1832–1845, 2023.

[11] J. Lee, M. Bjelonic, A. Reske, L. Wellhausen, T. Miki, and M. Hutter, "Learning robust autonomous navigation and locomotion for wheeled-legged robots," *Science Robotics*, vol. 9, no. 89, p. eadi9641, 2024.

[12] Q. Yuan, Z. Cao, M. Cao, and K. Li, "Reasan: Learning reactive safe navigation for legged robots," *arXiv preprint arXiv:2512.09537*, 2025.

[13] J. Wasserman, A. Agarwal, R. Jangir, G. Chowdhary, D. Pathak, and A. Gupta, "Legolas: Deep leg-inertial odometry," in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270. PMLR, 2025, pp. 2928–2947. [Online]. Available: https://proceedings.mlr.press/v270/wasserman25a.html

[14] C. Luo, Y. Wang, W. Cai, and M. Zhao, "Autoodom: Learning auto-regressive proprioceptive odometry for legged locomotion," in *2025 IEEE International Conference on Robotics and Biomimetics (ROBIO)*, 2025, pp. 2301–2306.

[15] A. Sadat, S. Casas, M. Ren, X. Wu, P. Dhawan, and R. Urtasun, "Perceive, predict, and plan: Safe motion planning through interpretable semantic representations," in *European Conference on Computer Vision*. Springer, 2020, pp. 414–430.

[16] M. Pfeiffer, M. Schaeuble, J. I. Nieto, R. Y. Siegwart, and C. Cadena, "From perception to decision: A data-driven approach to end-to-end motion planning for autonomous ground robots," *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1527–1533, 2016.

[17] D. Shah, A. Sridhar, A. Bhorkar, N. Hirose, and S. Levine, "Gnm: A general navigation model to drive any robot," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 7226–7233.

[18] D. Shah, A. K. Sridhar, N. Dashora, K. Stachowicz, K. Black, N. Hirose, and S. Levine, "Vint: A foundation model for visual navigation," in *Conference on Robot Learning*, 2023.

[19] F. Yang, C. Wang, C. Cadena, and M. Hutter, "iplanner: Imperative path planning," in *Robotics: Science and Systems*, 2023.

[20] P. Roth, J. Nubert, F. Yang, M. Mittal, and M. Hutter, "Viplanner: Visual semantic imperative learning for local navigation," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 5243–5249.

[21] D. Hoeller, L. Wellhausen, F. Farshidian, and M. Hutter, "Learning a state representation and navigation in cluttered and dynamic environments," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 5081–5088, 2021.

[22] C. Zhang, J. Jin, J. Frey, N. Rudin, M. Mattamala, C. Cadena, and M. Hutter, "Resilient legged local navigation: Learning to traverse with compromised perception end-to-end," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 34–41.

[23] T. He, C. Zhang, W. Xiao, G. He, C. Liu, and G. Shi, "Agile but safe: Learning collision-free high-speed legged locomotion," in *Robotics: Science and Systems*, 2024.

[24] P. Roth, J. Frey, C. Cadena, and M. Hutter, "Learned perceptive forward dynamics model for safe and platform-aware robotic navigation," in *Robotics: Science and Systems*, 2025.

[25] S. Chen, M. Yang, H. Mao, J. Zhang, H. Liu, S. He, D. Zhang, Z. Qiu, and C. Zhang, "Sea-nav: Efficient policy learning for safe and agile quadruped navigation in cluttered environments," 2026.

[26] P. Long, W. Liu, and J. Pan, "Deep-learned collision avoidance policy for distributed multiagent navigation," *IEEE Robotics and Automation Letters*, vol. 2, no. 2, pp. 656–663, 2017.

[27] P. Long, T. Fan, X. Liao, W. Liu, H. Zhang, and J. Pan, "Towards optimally decentralized multi-robot collision avoidance via deep reinforcement learning," in *2018 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2018, pp. 6252–6259.

[28] W. Ding, S. Li, H. Qian, and Y. Chen, "Hierarchical reinforcement learning framework towards multi-agent navigation," in *2018 IEEE international conference on robotics and biomimetics (ROBIO)*. IEEE, 2018, pp. 237–242.

[29] G. Welch, G. Bishop *et al.*, "An introduction to the kalman filter," 1995.

[30] M. Bloesch, C. Gehring, P. Fankhauser, M. Hutter, M. A. Hoepflinger, and R. Siegwart, "State estimation for legged robots on unstable and slippery terrain," in *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2013, pp. 6058–6064.

[31] M. Maravgakis, D.-E. Argiropoulos, S. Piperakis, and P. Trahanias, "Probabilistic contact state estimation for legged robots using inertial information," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2023, pp. 12 163–12 169.

[32] C. Chen, X. Lu, A. Markham, and N. Trigoni, "Ionet: Learning to cure the curse of drift in inertial odometry," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 32, no. 1, 2018.

[33] G. Cioffi, L. Bauersfeld, E. Kaufmann, and D. Scaramuzza, "Learned inertial odometry for autonomous drone racing," *IEEE Robotics and Automation Letters*, vol. 8, no. 5, pp. 2684–2691, 2023.

[34] M. Lauri, D. Hsu, and J. Pajarinen, "Partially observable Markov decision processes in robotics: A survey," *IEEE Transactions on Robotics*, vol. 39, no. 1, pp. 21–40, 2023.

[35] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[36] J. Long, Z. Wang, Q. Li, L. Cao, J. Gao, and J. Pang, "Hybrid internal model: Learning agile legged locomotion with simulated robot response," in *International Conference on Learning Representations*, vol. 2024, 2024, pp. 14 084–14 100.

[37] S. Mysore, G. Cheng, Y. Zhao, K. Saenko, and M. Wu, "Multi-critic actor learning: Teaching rl policies to act with style," in *International Conference on Learning Representations*, 2022.

[38] L. Wang, K. Yao, Y. Liu, W. Qin, J. Wu, Z. Sun, and Q. Zhu, "Puma: Perception-driven unified foothold prior for mobility augmented quadruped parkour," *arXiv preprint arXiv:2601.15995*, 2026.

[39] J. Schulman, P. Moritz, S. Levine, M. I. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," *CoRR*, vol. abs/1506.02438, 2015.

[40] R. Tarjan, "Depth-first search and linear graph algorithms," *SIAM Journal on Computing*, vol. 1, no. 2, pp. 146–160, 1972.

[41] C. Bai, T. Xiao, Y. Chen, H. Wang, F. Zhang, and X. Gao, "Faster-LIO: Lightweight tightly coupled lidar-inertial odometry using parallel sparse incremental voxels," *IEEE Robotics and Automation Letters*, vol. 7, no. 2, pp. 4861–4868, 2022.